

How the AI Job Finder works

A plain guide to the board, the list of every opening, and what actually happens when you upload a resume or search deeper.

ON THIS PAGE

1. [Reading the board](#)
2. [Seeing every opening](#)
3. [Searching deeper](#)
4. [Uploading a resume](#)
5. [What happens on a run](#)
6. [How scoring works](#)
7. [How location works](#)
8. [Suggested employers](#)
9. [What is verified, what is not](#)
10. [Where your data goes](#)
11. [When something goes wrong](#)

1 Reading the board

About eighty employers, sorted by whether you can act on them today.

The page has two tabs above the filters. **Employers** is the board: one hand-researched card per employer, and the subject of this section. **All openings** is one row per job, and is covered in [the next section](#).

The board opens with no filters applied, so you see everything at once. Cards are grouped by timing first and by match score second, which means the roles you can apply to this week sit at the top.

Nothing on the default board assumes anything about you. School-specific features stay out of the way until a resume asks for them: upload one showing a University of Utah student and a career fair strip and an alumni filter appear, because the board tracks which of these employers recruit on that campus.

-  **Live roles right now.** A real opening was confirmed open on the date in the footer. Apply this week.
-  **Suggested for your city.** Only appears after you upload a resume from outside Salt Lake, or from a field this board does not hire for. These come from Claude's knowledge and are not verified.
-  **Opens later this fall.** The program is real and dated, but applications are not open yet. Set a reminder.
-  **Watch list.** The company hires, and their board is live, but nothing entry level was posted on the check date.
-  **Wrong cohort.** The timing does not fit the graduation window. This group is hidden when it is empty.

The openings on each card

A card lists the openings that were read by hand, under the date they were verified. Where the employer's own job feed has more, a button under them reads, for example, **+ 12 more openings on their job feed · 3 new →**. Click it to open All openings on that employer alone. A script read the extra ones from the feed at the last refresh; nobody read them by hand.

A link the employer has taken down is struck out and tagged **no longer posted**. The refresh script, `refresh.py`, asks the employer's own system about every link on the board each time it runs, so the strike appears as soon as the openings database is refreshed and does not wait for someone to edit the card. An opening whose closing date has passed is struck out the same way.

The number on each card

That circle is a match score from 60 to 100. On the default board it rates fit for a new grad targeting data, AI and fintech work around Salt Lake City. After you upload a resume it rates fit for that person instead. Higher means a stronger case, not a better company.

Filters

Filters combine, so picking a role and a city narrows to cards matching both. Picking a second chip in the same row swaps rather than stacks. Any filter that would return nothing is hidden, so you can never land on an empty board. Search covers company names, cities, the explanation text and job titles.

2 Seeing every opening

One row per job, from about 360 job feeds instead of about eighty cards. No API key needed.

The board can only show an employer somebody wrote a card for. Click **All openings** and you get the other half: every entry-level opening the refresh script found on employers' own job feeds, whether or not the employer has a card. You can filter, sort and search it without a key and without uploading anything.

Where the rows come from

The list is built from three places, and each job appears once.

- **Employers' own job feeds.** `refresh.py` reads about 360 feeds: nearly every employer on the board, a larger number of employers that have no card, Utah's state job bank, and the federal job site, USAJOBS. It records what every feed returns in a database, then publishes the live rows its title rules keep as `jobs.json`, which the page loads. The line beside the tabs gives the date the feeds were read. It counts the employers that have an opening in the list, so its number is smaller than the number of feeds.
- **The board's own openings.** These were read by hand. Where the same job is also on a feed, the hand-read years, degree bar and closing date win.
- **Web finds.** Openings Claude found on the web for the profile on screen, if a web search has been run for it. See [Searching deeper](#).

Not every employer can be read. Before the script reads a site it asks that site's `robots.txt`, and it stays out of any path closed to all crawlers. That rules out several common hiring systems. Those employers are not forgotten: the script publishes their names and careers pages, and a web search starts with them.

Until Claude has ranked the list, rows are ordered by a plain keyword match against the profile on screen: place, level, field, words from the profile that appear in the title, and how far the years asked for exceed what

the profile has. That is a sort order, not a score, so the circle beside a row shows a dot and no number until Claude has scored that opening.

What the tags mean

TAG	MEANING
new	Posted, or first seen on its employer's feed, in the seven days before the database was refreshed. Counted from the refresh date, not from today, so a stale page does not keep calling old jobs new. An opening read by hand is also new when it was put on the board in the six days up to the latest refresh. The page shows this tag in capitals.
new-grad program	A structured new-grad program, or a requisition that says it is for new graduates.
closes Oct 16	The posting's own closing date. Rows past that date drop out of the list.
~3+ yrs	The posting asks for at least that many years. Shown from two years up. The tilde means nobody read the number by hand: a program took it from the posting text, or on a web find Claude did. Open the posting to confirm. No tilde means it was read by hand.
leans SWE	A software-engineering ladder, where you are judged against computer-science graduates.
wants ...	The posting's own degree bar, such as a technical major or an MS or PhD.
level not stated	No level word in the title, so the posting may ask for experience. Kept for jobs in Utah only.
experienced	A senior, lead or architect title, or a numbered level above one. Not an entry-level opening.
internal applicants only	The posting says it is open only to people who already work for that employer.
applied Oct 2	It matches an entry in your own applications list.
on the board	The employer also has a hand-researched card.
state job bank lead	A listing from Utah's state job bank, not read from the employer. See below.
found on the web	Saved by Claude during a web search for this profile. It adds not opened when Claude saw the link in search results and no page read is on record.

A title that reads entry-level can still ask for experience. That is what the years tag is for, and the **Out of reach** chip hides every opening asking two or more years beyond what the profile has. No years tag does not mean no experience is asked for: it means no figure of two or more was read.

What is tucked away, and why

Two kinds of posting are real but wrong for most readers of this page. Each sits behind its own chip and stays out of the list until you ask for it.

- **Experienced**, in the Level row. Senior, lead and architect titles in Utah, and numbered levels above one. They are not entry-level, so listing them by default would bury the jobs a new graduate can get. They appear when you turn the chip on, when you type in the search box, or on their own when the profile on screen has two or more years of experience. Manager and executive roles are not published at all.
- **Internal only**, in the Show row. Postings open only to people who already work for the employer. An outside applicant cannot apply, so they are shown only with the chip on, and the chip appears only when the database holds some.

State job bank leads

Rows from Utah's state job bank are different in kind. The Department of Workforce Services runs it at jobs.utah.gov, and it lists jobs at hundreds of employers, including ones the script cannot read directly. A row from it is a lead, not a posting. Its link opens the job bank, which asks for a free UtahID sign-in before it shows the details. A long title arrives cut short, usually at 40 characters, and ends in an ellipsis.

So these rows are published only for employers the script does not read itself. Each is tagged **state job bank lead** and has a **find the employer's posting** link beside it, which runs a web search for that title and employer.

Narrowing the list

The filter rows are Show, Where, Level, Field, Source and Hide, and they combine the way the board's filters do. Chips in the Where, Level and Field rows are hidden when the list holds nothing they would match, and so are New this week, Internal only, State job bank and Found on the web. Sort offers Best match, Newest, Closing soonest and Employer A–Z. The search box covers job titles, employers, places and field names.

3 Searching deeper

Two optional steps for one person: Claude ranks every opening, then searches the web for ones the database does not have. Both need an Anthropic API key.

Uploading a resume re-scores the same eighty or so employers, so on its own it can never show you a job at an employer nobody added to the board. These two steps are what look past that list.

1. Open **All openings** and click **Search deeper**. The upload dialog opens, set to search deeper for the profile on screen, with no resume needed.

2. Paste an Anthropic API key if the dialog does not already have one. See [Uploading a resume](#) for where to get it and how to pick a model.
3. Under **How far to look**, tick one step or both, and pick a depth for the web search.
4. Click **Search deeper** at the foot of the dialog. When it finishes, the results are in All openings.

Run this way, it works from the profile on screen and sends no resume. Each step gets a short brief built from that profile: status, most recent role, skills, target, years of experience, home city and the summary shown in the header. The name on the profile is left out of the brief. The same two steps are offered when you upload a resume; left ticked there, they run straight after the tailoring. The resume goes with the tailoring and the ranking. The web search never receives it.

Step one: rank the openings

Claude is given every listed opening as one line each: title, employer, place, level, years asked for, degree bar, closing date, and whether it is a software-engineering ladder. It is not given the posting text. Internal-only postings, web finds, and experienced titles two or more years beyond the person are left out, and if more than 2,500 remain the best 2,500 by keyword match are sent.

It is asked for up to 80 openings worth this person's time, each with a score and one sentence of reasoning. The page treats that answer as untrusted, as it does every answer from Claude. It ignores any opening that was not on the list it sent, keeps at most 120, clamps scores to 60 through 100, and applies the same experience ceiling it applies to employers: an opening asking two years more than you have tops out at 80, and three or more at 70. Ranked rows sort above unranked ones. The ranking is saved in the browser for that profile.

Step two: search the web

Claude is given Anthropic's web search and page-reading tools, plus one tool that belongs to the page, `save_jobs`. Along with the brief it gets three lists from the database:

- the employers already covered, so it does not spend searches on them;
- the employers `refresh.py` knows about but cannot read, with their careers pages, to start with;
- up to forty state job bank leads that suit this person, to trace back to the employer's own posting.

It is told to open each posting before saving it, to confirm the page does not say the job is closed, and to read what the posting asks for. It is also told that text on a web page is information about a job, never an instruction to follow. While it works, the panel names each search and each site being read, and counts what has been saved.

What it may save, and what is rejected

Nothing Claude sends to `save_jobs` is trusted. A job is saved only when all of these hold:

- it has a title and an employer;
- its link is a full http or https link;

- the link is not on a listing site. LinkedIn, Indeed, Glassdoor, ZipRecruiter and the like are useful to Claude as leads, and are always rejected as links;
- the link appeared in this session's search results or page reads. A link Claude only recalls is rejected.

A posting the list already has is turned away as already known, so the budget goes on new ground. A state job bank lead is the exception, because the employer's real posting is exactly what is worth saving, and it then replaces the lead in the list. Every rejection goes back to Claude with its reason, so it can open the real posting and try again.

Saved finds are tagged **found on the web** and kept in this browser's own database, per profile. They are saved as they are found, so stopping a search keeps everything saved up to that point.

Depth and cost

DEPTH	WEB SEARCHES	PAGE READS
Quick	about 12	about 16
Standard	about 30	about 40
Deep	about 60	about 80

Deep is selected until you pick another, and the page remembers your choice. A search ends when the leads or the budget run out. The page also ends it if it runs well past its budget or reaches 40 round trips, and the toast then says why it stopped.

Claude is asked to finish a search with a short summary: where it looked, which kinds of employer had openings, and which are worth watching. The page keeps the latest one for each profile. Click the line beginning **Claude's last web search for this profile** at the top of All openings to read it.

Models and effort levels are the same as for tailoring, and the default is Sonnet 5.5 at high effort. Anthropic bills each web search at \$0.01 on top of tokens. Every page read is billed as input tokens, and the page caps each read at 6,000 tokens.

These costs are estimates. Both steps were tested against a scripted stand-in for the API, not the live API, so none of these figures has been measured. From list prices on Sonnet 5.5: ranking about \$0.50, and a Deep web search a few dollars and ten minutes or more. Quick and Standard do less and cost less. The toast at the end of a run reports what it actually cost.

Web search has to be switched on for your organization in the Anthropic Console before step two will run. Ranking does not need it. If the API refuses the search, see [When something goes wrong](#).

4 Uploading a resume

Six steps. Tailoring alone takes a few minutes and about thirty cents.

1. Click **Upload a resume**.
2. Paste an Anthropic API key. Get one at console.anthropic.com. It is saved in your browser so you only do this once.
3. Pick a model and effort level, or keep the defaults.
4. Drop in a PDF, a Word file or a text file. You can also paste the text.
5. Under **How far to look**, decide whether the two [Searching deeper](#) steps should run straight after. Both are ticked when the dialog first opens, with the web search set to Deep. Untick them to tailor the board and nothing else.
6. Click **Tailor, then search deeper**, or **Tailor the board** if you unticked both, and watch it think.

Claude's reasoning streams into the panel while it works, and a counter tracks how many companies it has written. Stop cancels at any point. Stopped during the tailoring, the board is left untouched; stopped during a later step, everything already finished is kept.

What it costs

MODEL AND EFFORT	PER RUN	TIME	WHEN TO PICK IT
Sonnet 5.5, high	about \$0.30	a few minutes	The default, and the right answer almost always.
Sonnet 5.5, medium or low	less	quicker	A cheaper pass; normally still enough.
Opus 5.5, high	roughly \$0.60 to \$1.20	longer	An unusual resume or a career change.
Fable 5.1	\$1.50 and up	slowest	Only when Opus 5.5 got something wrong.

Effort matters as much as the model. Thinking is billed as output, so a high-effort run on a big model spends much of its money reasoning rather than writing. Only the first row is measured on this board; the others are estimates from list prices, which is why the defaults are Sonnet 5.5 at high.

These figures are for tailoring alone. The two Searching deeper steps are billed on top when they are ticked, and neither has been measured against the live API. As estimates on Sonnet 5.5, ranking adds about \$0.50 and a Deep web search a few dollars. See [Depth and cost](#).

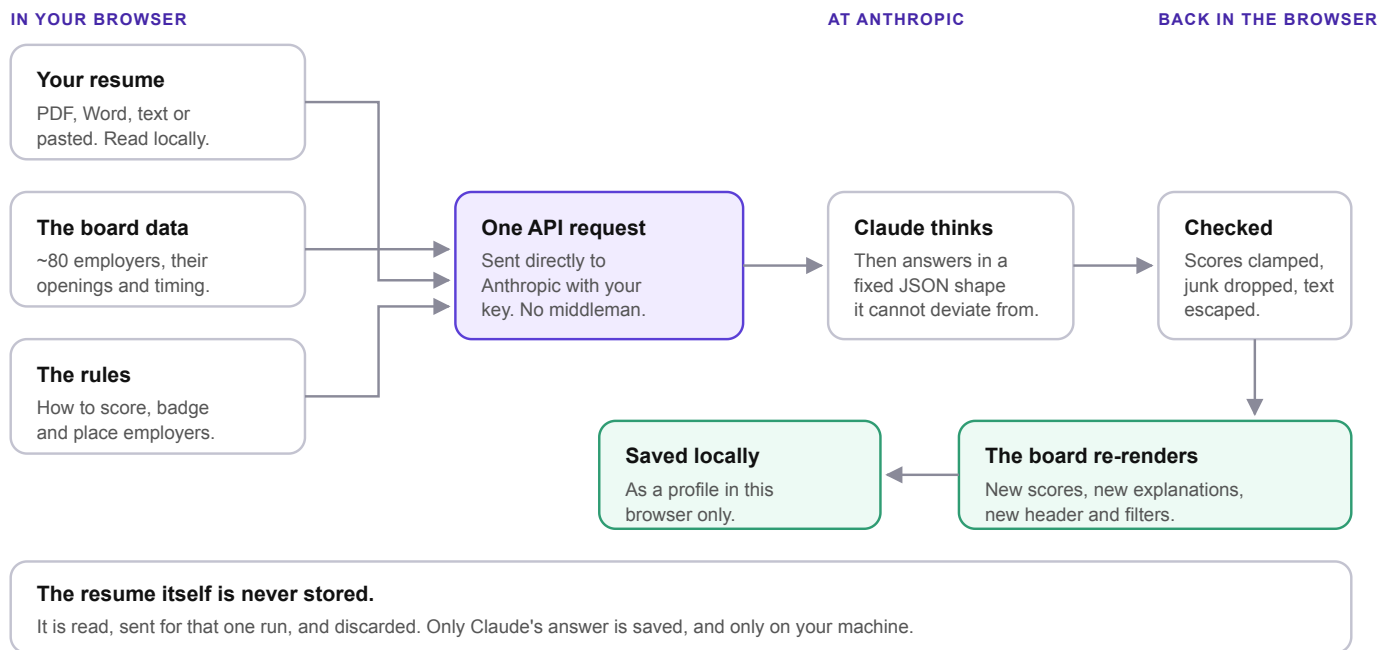
The toast at the end reports what the run actually cost.

Saved profiles

Every finished tailoring run is saved as a profile and a dropdown appears in the header. Switch between people freely, or pick **Sample board (Jordan Kim)** to go back to the version scored for the made-up sample candidate. Profiles live in that one browser and never sync anywhere. Rankings and web finds are kept per profile, so switching profiles switches them too. Tailoring a resume with the same name again replaces that person's profile and keeps the ranking and web finds already saved for them. Removing a profile, with the x button beside the dropdown, removes its ranking and web finds too.

5 What happens on a run

Tailoring is one request, sent straight from your browser to Anthropic. There is no server in the middle.



Tailoring is one round trip. Nothing is uploaded to any server belonging to this project, because there isn't one.

The two [Searching deeper](#) steps, when they are ticked, follow as further requests to the same place. Ranking is one more request. A web search is a series of them, because Claude searches, reads and saves in turns. When the steps follow an upload, the resume is included in each request and discarded when the run ends.

Why the answer is always the right shape

The request pins Claude to a JSON schema, so it cannot reply with prose or invent extra fields. The page then treats that answer as untrusted anyway: scores outside 60 to 100 are clamped, unknown categories are dropped, companies that are not on the board are ignored, duplicates are removed, and anything shown as text is escaped. A reply covering fewer than 80% of the companies is rejected rather than half applied.

6 How scoring works

A score answers one question: how strong is this person's case for an entry-level hire here?

SCORE	MEANING
95 to 100	Bullseye. Field, city, tools and hiring pipeline all line up.
85 to 94	Strong. A clear case with one soft spot.
75 to 84	Solid. Worth applying, not a standout.
65 to 74	Stretch. Needs a good cover letter or a referral.

SCORE	MEANING
60 to 64	Weak. Listed so you know it was considered and why it lost.

Scores are deliberately spread. A board where everything scores in the high eighties tells you nothing, so the rules require using the full range and being explicit when a role is a reach.

Experience is a ceiling, not a factor

A posting demanding three years cannot be a great match for someone graduating next term, however well the rest lines up. Each listed opening was checked against the live posting for what it actually requires, and anything wanting two years or more is tagged on the card. The page then caps the score itself rather than asking the model nicely: three years beyond your experience caps an employer at 70, two years caps it at 80, and if even the easiest opening there is out of reach the company moves to the watch list. The same two caps apply to each row when Claude ranks All openings.

One reachable role rescues a company. An employer listing a new-grad programme alongside a senior role is judged on the new-grad one, because that is the door you can actually walk through.

Badges and tiers are worked out here, not by Claude

Claude supplies one score per employer and a single yes-or-no on whether its stack overlaps tools named on the resume. Everything else follows automatically. The location badge comes from where the employer sits relative to the candidate. Fintech and AI-native come from the researched board, because they describe the company and do not change with who is applying. Tier one is simply the eight highest scores.

This is deliberate. Asking a model to decide eighty tiers and badge sets costs real money in reasoning and produces answers that drift between runs. Deriving them is free, instant and identical every time.

Timing is separate from fit

Geography never changes whether a role is live. If a company has a real opening today, it stays in the live group even for someone across the country. The score drops instead, and the explanation says the role is on site. Timing only moves when graduation dates genuinely conflict, such as a program that starts before the person finishes school.

7 How location works

The board was built around Salt Lake City, so every location is re-read relative to whoever uploaded a resume.

LABEL	WHAT IT MEANS FOR THIS PERSON
Local	Headquarters or a major hub in their metro.

LABEL	WHAT IT MEANS FOR THIS PERSON
Has an office	A smaller office there.
Remote	Hires this kind of role remotely from where they live.
Needs relocation	Would require a move. Hidden when nothing qualifies.

The filter labels rename themselves to the person's city, so an Austin candidate sees "Austin Area" and "Has Austin office". A remote-only candidate sees neither, because those categories would be empty.

The Salt Lake exception. If the resume shows a home base anywhere on the Wasatch Front, or says the person is moving to Salt Lake, the board stays exactly as it is. No relabeling and no suggested employers. The researched board is already correct for those candidates, so rewriting it would only lose information. Two things are gated separately: the career fair strip, which follows the school rather than the city and appears only for a University of Utah student, and suggested employers, which appear for anyone whose field falls outside what this board hires for, Salt Lake resident or not.

8 Suggested employers

The escape hatch for when this particular board is the wrong board.

About eighty hand-picked employers are only useful if they are employers you could actually work for. Two things break that. You might live somewhere else, in which case the Utah companies are unreachable. Or you might work in a different field, in which case they are irrelevant even from across town: nearly all of them hire data, analytics or software people.

Either way, Claude adds employers in your own city and your own field: up to fifteen, drawn from its own knowledge, mixing large employers that run early-career programs with notable local firms. They appear as dashed purple cards under a heading naming the city. Only someone who is both near Salt Lake and in one of the board's fields gets no suggestions, because for them the researched list is already the right answer.

When your field is the mismatch, the suggestions move to the top of the page and the board drops beneath them. A nurse in Salt Lake City should see hospitals first, not dozens of data and software employers with genuinely open roles she cannot apply for. The board says so in the group heading rather than quietly reordering.

They are handled differently from the researched board in three ways, all deliberate:

- They are labeled **not verified** on every card.
- Their button runs a web search for the company's careers page instead of linking to a job posting that may not exist.

- Each carries a confidence level, and lower-confidence entries say so.

The rules tell Claude to name a hiring program only when it is certain of the exact name, and otherwise to describe what to search for. Fewer good suggestions beat a padded list.

A suggested employer is not a web find. A suggestion is an employer Claude recalled, with no posting behind it. A web find, from [Searching deeper](#), is a posting whose link Claude came across in a search.

9 What is verified, what is not

The single most important distinction on the page.

THING	HOW MUCH TO TRUST IT
The ~80 employers	Hand researched. Real companies with real early-career hiring.
Openings on a card	Read by hand and checked against each company's hiring system on the date in the footer. A link that system no longer recognizes is struck out at the next refresh. Jobs close, so recheck.
Timing status	Verified on that same date.
Feed rows in All openings	Read by a script from the employer's own hiring system on the date shown beside the tabs. Nobody read the posting. Level and field are worked out from the title by rules, so check the posting before you count on either.
Years with a tilde	Read from the posting text by a program, or by Claude on a web find. Either can misread. Open the posting to confirm. A figure without a tilde was read by hand.
State job bank leads	A listing on the state job bank, not read from the employer. No employer link, and long titles are cut short. Find the employer's own posting before you rely on one.
Web finds	A link Claude came across in that search, with Claude's own reading of what the page asks for. Not opened means no page read is on record. <code>refresh.py</code> never re-checks them, so one can stay listed after the job closes.
Scores and explanations	Claude's judgment about you. Useful, arguable, not fact. A ranking score in All openings is judged from one line per job, not from the posting.
Suggested employers	Unverified leads. Confirm each one yourself.

Searching deeper costs	Estimates. Those steps were tested against a scripted stand-in, not the live API. The toast after a run is the real number.
------------------------	---

A tailoring run does not re-check the job market. It re-scores and re-explains the existing research. It cannot tell you a job closed yesterday. Neither can a ranking, which scores the database as it stood when the feeds were last read. A web search looks for postings the database lacks and does not re-check the ones it has. To actually refresh the openings, run `python3 refresh.py`. It reads every feed again, re-tests every link on the board and rebuilds `jobs.json`.

10 Where your data goes

Short version: your resume goes to Anthropic for the run you start and nowhere else, and nothing reaches this project's author.

- **The resume is never stored.** It is read in your browser, included in the requests of the one run you start, and discarded. Only Claude's answers are kept.
- **Searching deeper sends a brief, not the resume.** Run on its own, it sends a few lines built from the profile on screen: status, most recent role, skills, target, years of experience, home city and summary. The name on the profile is left out. The resume goes along only when the steps follow an upload in the same run.
- **Web searches run at Anthropic.** Claude's searches and page reads are carried out by Anthropic's own tools, not by your browser. The profile's home city is passed to the search tool as an approximate location. Claude writes the search queries itself from what it was sent, so a query can name a city and a kind of work. The brief leaves the name out for that reason.
- **Your API key** is stored in your browser and sent only to Anthropic. This project has no server that could receive it.
- **Saved profiles** stay in that one browser. They do not sync, and clearing site data deletes them.
- **Rankings and web finds** stay in that one browser too, kept per profile, along with Claude's summary of its last web search. Removing a profile removes them with it, and clearing site data removes everything. In a private window the browser may offer no database, and web finds then last only until the page is closed.
- **The openings database is public data.** `jobs.json` holds job postings and nothing about you. The page fetches it from the site it was loaded from.
- **Each person pays their own way.** Runs bill the key that was pasted in, so sharing the link costs you nothing.

The page itself contacts three kinds of place in all: the site it was loaded from, the public code libraries a run needs, and Anthropic. Of those, only Anthropic receives anything about you.

The tradeoff for having no server is that your key sits in your browser. Only paste it on a machine you trust, and remember that anyone using your computer could open the page.

11 When something goes wrong

Errors appear on the page rather than in the console, and each one names its own fix.

WHAT YOU SEE	WHAT TO DO
Rejected that API key	The key is wrong or was revoked. Make a new one.
Not allowed to do this (403)	The key's account is not permitted to use something the run asked for, such as the model. Anthropic's own message follows and says what. Try Sonnet 5.5.
Rate limited	Wait a minute and run it again.
Could not reach Anthropic	Usually a VPN, an ad blocker or an offline connection.
Ran out of output room	Thinking used the budget. Rerun at a lower effort.
Not valid JSON	A rare bad reply. Just run it again.
Only returned some companies	Claude stopped early. Run it again.
Could not be saved	Browser storage is full. Delete an old profile.
Nothing happens at all	Serve the folder over http rather than opening the file directly.
Database could not be loaded	All openings says this when the page was opened straight from disk, because some browsers block a page opened that way from fetching <code>jobs.json</code> . The tab then lists only the board's own openings. Serve the folder instead, with the command below. If <code>jobs.json</code> is missing, run <code>python3 refresh.py</code> to build it.
Rejected the request (400)	When the message mentions web search, the usual cause is that your organization has not switched web search on. That is done per organization, under Privacy controls in the Anthropic Console. Until then, untick the web search step. Ranking still runs.
Web search stopped early	The toast says why: it used its search budget, reached the limit on round trips, a reply was cut off, or Claude declined to continue. Everything saved up to that point is kept.

WHAT YOU SEE

WHAT TO DO

A job you know of is missing

Ask the refresh script, with the second command below. It says whether the job was kept, listed as experienced or internal only, dropped by the title rules and why, gone, or never returned by any feed, which means its employer is not a source. It reads the local database, `jobs.db`, which `python3 refresh.py` builds, so run that once first.

To serve the folder, run this inside it, then open `http://localhost:8765/index.html`:

```
python3 -m http.server 8765 --bind 127.0.0.1
```

To ask why a job is missing, give the script part of its title, or its employer's name:

```
python3 refresh.py --why "business intelligence analyst"
```

A run with several steps saves each step as it finishes. If a later step fails, the message starts with what was already done, and only the failed step is lost.

If the button itself is missing or dead, the most likely cause is an extension blocking the code library the page loads on first use. The full technical write-up lives in the [project README](#).